CODE A NOVA

15-Day Internship Project Brief • Approved by HR

MACHINE LEARNING

MediScan — Medical Image Classification System

Build a deep learning-based medical image classification system that can detect conditions from X-ray/skin lesion images using transfer learning (ResNet/EfficientNet), with data augmentation, model evaluation, explainability (Grad-CAM), and a Flask/Streamlit web interface for predictions.

■ Domain Overview

Domain: Machine Learning focuses on building systems that learn patterns from data to make predictions or decisions. This domain covers supervised/unsupervised learning, deep learning, model training and evaluation, hyperparameter tuning, and deploying ML models to production.

Objective: Build and deploy a medical image classification system (MediScan) using transfer learning on a pre-trained CNN, with proper data preprocessing, augmentation, training pipeline, comprehensive evaluation, model explainability, and a web-based prediction interface.

Expected Outcome: A trained deep learning model with > 85% accuracy on a medical imaging dataset, with Grad-CAM explainability, comprehensive evaluation metrics, and a deployed web interface for uploading and classifying images.

■■ Tech Stack

Category	Technology
Language	Python 3.11+
DL Framework	PyTorch or TensorFlow/Keras
Pre-trained Models	ResNet50, EfficientNet-B0 (torchvision/tf.keras)
Data Processing	Pandas, NumPy, Pillow, OpenCV
Augmentation	Albumentations or torchvision.transforms
Visualization	Matplotlib, Seaborn
Explainability	Grad-CAM (pytorch-grad-cam or custom)
Web Interface	Streamlit or Flask

Experiment Tracking	MLflow or Weights & Biases (free tier)
Deployment	Streamlit Cloud or HuggingFace Spaces

■■ Architecture & Data Flow

- **Pipeline:** Raw Images → Preprocessing → Augmentation → DataLoader → Model → Predictions
- **Model:** Pre-trained CNN (ResNet50/EfficientNet) → Custom classifier head → Fine-tuned
- **Training:** Transfer learning (freeze base → train head → gradual unfreezing → fine-tune all)
- **Evaluation:** Train/val/test split (70/15/15), metrics per class, confusion matrix
- **Explainability:** Grad-CAM heatmaps overlaid on input images to show decision regions
- **Inference:** Web upload → Preprocess → Model predict → Class + confidence + Grad-CAM → Display

■ 15-Day Milestone Plan

Day	Goals & Tasks
Day 1	Project setup, dataset selection (Chest X-ray or Skin Lesion from Kaggle), initial exploration
Day 2	Data preprocessing — Resize, normalize, train/val/test split, class distribution analysis
Day 3	Data augmentation — Random flips, rotations, color jitter, affine transforms; visualize samples
Day 4	DataLoader setup — Custom dataset class, batching, efficient loading with prefetch
Day 5	Base model setup — Load pre-trained ResNet50/EfficientNet, modify classifier head
Day 6	Training loop — Loss function (CrossEntropy), optimizer (Adam), learning rate scheduler
Day 7	Training execution — Train for 10-20 epochs, monitor train/val loss and accuracy curves
Day 8	Hyperparameter tuning — Learning rate, batch size, augmentation strength, dropout rate
Day 9	Model evaluation — Accuracy, precision, recall, F1 per class, confusion matrix, ROC curves
Day 10	Grad-CAM implementation — Generate heatmaps for sample predictions, overlay on images
Day 11	Model comparison — Train a second architecture, compare metrics side-by-side
Day 12	Web interface — Streamlit/Flask app with image upload, prediction display, Grad-CAM view
Day 13	Experiment tracking — Log metrics, hyperparams, artifacts with MLflow/W&B;
Day 14	Edge case analysis — Misclassified samples, failure modes, bias detection
Day 15	Deploy web app, prepare model card, demo with sample images, final presentation

■ Deliverables

1. Training pipeline code (notebooks + scripts)
2. Trained model weights (.pt / .h5 file)
3. Deployed web application URL
4. Model evaluation report with all metrics and visualizations
5. Grad-CAM explainability samples (10+ images with heatmaps)
6. Model card (dataset, architecture, performance, limitations, ethical considerations)
7. 7-minute presentation with live classification demo

■ Evaluation Rubric

Criteria	Weight	Success Indicators
Model Performance	25%	Accuracy > 85%, balanced per-class metrics, proper evaluation methodology
ML Engineering	20%	Training pipeline quality, data handling, augmentation, hyperparameter tuning
Explainability	15%	Grad-CAM quality, interpretation of results, failure case analysis
Code Quality	15%	Modular code, reproducibility, experiment tracking, documentation
Web Application	15%	Upload/predict flow, result display, Grad-CAM visualization, usability
Presentation	10%	Live demo, model card quality, ethical considerations, Q&A;
TOTAL	**100%**	

■ Feature List & Wireframe Notes

■ Image upload and classification with confidence scores
■ Grad-CAM heatmap visualization for model interpretability
■ Support for multiple image formats (JPEG, PNG, DICOM conversion)
■ Batch prediction mode for multiple images
■ Training/validation loss and accuracy curves

■ Per-class precision, recall, F1 scores with confusion matrix
■ ROC-AUC curves for each class
■ Model comparison dashboard (two architectures)
■ Experiment history with hyperparameter logging
■ Download prediction results as CSV

■ Resources & References

- PyTorch Tutorials: https://pytorch.org/tutorials/
- TensorFlow Guides: https://www.tensorflow.org/guide
- Grad-CAM Paper: https://arxiv.org/abs/1610.02391
- Kaggle Medical Datasets: https://www.kaggle.com/datasets?tags=health
- MLflow: https://mlflow.org/docs/latest/

■■ Constraints & Guidelines:

- Must use a publicly available medical imaging dataset
- Training must be feasible on Google Colab free GPU (T4)
- Model inference must complete in < 3 seconds per image
- Must include ethical considerations and limitations in model card